

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Ashika.B	Reg.No.:	192521336
Department:		Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I Ashika.B(192521336) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Pseudocode Writing Task – Algorithm Design

1. DMA-Based Data Transfer from I/O Device to Memory

Aim

To design a DMA-based data transfer algorithm that transfers data from an I/O device to memory while minimizing CPU involvement and handling interrupts efficiently.

Pseudocode

ALGORITHM DMA_Data_Transfer

BEGIN

 Initialize I/O device

 Initialize DMA controller

 Set source_address $\leftarrow$ I/O device

 Set destination_address $\leftarrow$ Memory address

 Set transfer_count $\leftarrow$ Number of bytes to transfer

 Configure DMA controller with:

 Source address

 Destination address

 Transfer count

 Transfer mode

 Enable DMA interrupt

 Request DMA controller to start transfer

WHILE transfer_count > 0 DO

 DMA controller transfers data

 DMA controller updates memory address

 DMA controller decreases transfer_count

END WHILE

Generate DMA interrupt

CPU receives interrupt

IF transfer completed successfully THEN

 Acknowledge interrupt

 Update transfer status

 Notify application

ELSE

 Handle DMA error

END IF

Disable DMA interrupt

Release DMA controller

END

Explanation

The CPU only initializes the DMA controller and starts the transfer. The DMA controller directly transfers data from the I/O device to memory. After the transfer is completed, the DMA controller generates an interrupt to inform the CPU. Thus, CPU involvement is minimized and system performance is improved.

2. Vectored Interrupt Handling for Multiple Devices

Aim

To design an interrupt handling system that prioritizes interrupts from multiple devices and dispatches the appropriate Interrupt Service Routine (ISR).

Pseudocode

ALGORITHM Vectored_Interrupt_Handler

BEGIN

 Initialize interrupt controller

 Define interrupt vectors:

 Device1 → ISR1

 Device2 → ISR2

 Device3 → ISR3

Device4 → ISR4

Assign priority to each device

Enable interrupts

WHILE system is running DO

IF interrupt request is received THEN

Identify interrupting device

Read interrupt vector

Determine priority of interrupt

IF interrupt has higher priority than current task THEN

Save CPU registers and current context

Select corresponding ISR

Execute ISR

Clear interrupt request

Restore CPU registers and previous context

Return from interrupt

ELSE

Store interrupt in pending queue

END IF

END IF

END WHILE

END

Explanation

When multiple devices generate interrupts, the interrupt controller identifies the source and checks its priority. The vectored interrupt provides the address of the appropriate ISR directly. The highest-priority interrupt is serviced first, after which the CPU returns to its previous task.

3. Dynamic Selection of Memory-Mapped I/O and I/O-Mapped I/O

Aim

To dynamically select between memory-mapped I/O and I/O-mapped I/O based on latency and bandwidth requirements.

Pseudocode

ALGORITHM Select_IO_Mapping

BEGIN

Read device requirements

Read latency_requirement

Read bandwidth_requirement

Read memory_availability

IF latency_requirement = HIGH AND

bandwidth_requirement = HIGH THEN

Select MEMORY_MAPPED_IO

Use normal memory instructions

Allocate memory address space for device

ELSE IF latency_requirement = LOW AND

bandwidth_requirement = LOW THEN

Select IO_MAPPED_IO

Use special I/O instructions

Allocate separate I/O address space

ELSE

IF available_memory_address_space is sufficient THEN

Select MEMORY_MAPPED_IO

ELSE

Select IO_MAPPED_IO

END IF

END IF

Configure selected I/O mechanism

Start data transfer

Monitor latency and bandwidth

IF performance is below requirement THEN

Re-evaluate I/O mapping

Switch to more suitable method

END IF

END

Explanation

Memory-mapped I/O allows devices to be accessed using normal memory instructions and is suitable when high performance is required. I/O-mapped I/O uses a separate I/O address space and specialized instructions. The algorithm dynamically selects the appropriate method according to system requirements.

4. Bus Arbitration for PCIe, USB and Thunderbolt

Aim

To design a bus arbitration algorithm that manages communication requests from multiple devices connected through PCIe, USB, and Thunderbolt interfaces.

Pseudocode

ALGORITHM Bus_Arbitration

BEGIN

 Initialize PCIe, USB and Thunderbolt interfaces

 Create request_queue

 WHILE system is running DO

 Check PCIe request

 Check USB request

 Check Thunderbolt request

 IF request is available THEN

 Add request to request_queue

 END IF

 FOR each request in request_queue DO

 Calculate priority based on:

 Interface type

 Data size

 Latency requirement

 Device priority

 END FOR

 Select highest-priority request

 IF selected request = PCIe THEN

 Grant PCIe bus access

 Transfer data

 Release PCIe resources

```
ELSE IF selected request = USB THEN
```

```
    Grant USB bus access
```

```
    Transfer data
```

```
    Release USB resources
```

```
ELSE IF selected request = Thunderbolt THEN
```

```
    Grant Thunderbolt bus access
```

```
    Transfer data
```

```
    Release Thunderbolt resources
```

```
END IF
```

```
Remove completed request from queue
```

```
END WHILE
```

```
END
```

Explanation

The algorithm continuously monitors PCIe, USB, and Thunderbolt devices for communication requests. Each request is assigned a priority based on factors such as latency, data size, and device importance. The highest-priority request receives access first, preventing conflicts and improving bus utilization.

5. Hybrid I/O Communication Using Polling and DMA

Aim

To design a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Pseudocode

```
ALGORITHM Hybrid_IO_Communication
```

```
BEGIN
```

```
    Initialize all I/O devices
```

```
    FOR each device DO
```

```
        Determine device speed
```

```
        IF device_speed = LOW THEN
```

```
            Select POLLING mode
```


WHILE data is required DO

 Check device status

 IF data is available THEN

 Read data from device

 Process data

 END IF

END WHILE

ELSE IF device_speed = HIGH THEN

 Select DMA mode

 Configure DMA controller

 Set source address

 Set destination memory address

 Set transfer size

 Enable DMA interrupt

 Start DMA transfer

 WHILE DMA transfer is in progress DO

 CPU performs other tasks

 END WHILE

 WAIT for DMA interrupt

 IF DMA interrupt received THEN

 Verify transfer

 Process received data

 Clear interrupt

 ELSE

 Handle transfer error

 END IF

END IF

END FOR

Continue normal system operation

END

Explanation

The hybrid mechanism chooses the I/O method according to device speed. **Low-speed devices** use polling because their data rate is small and the implementation is simple. **High-speed devices** use DMA, allowing data to move directly between the device and memory while the CPU performs other tasks. An interrupt informs the CPU when the DMA transfer is complete.